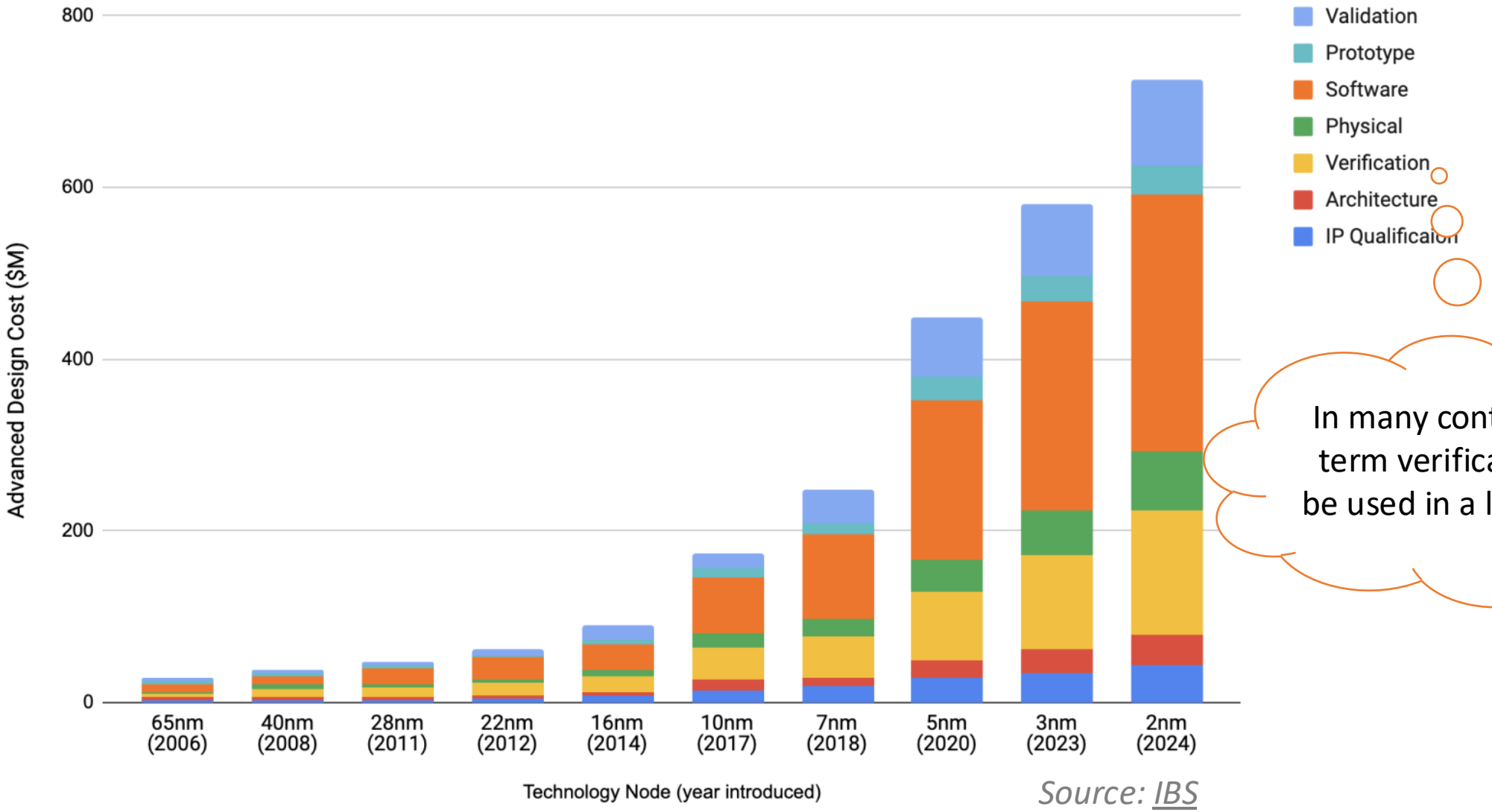
Formal Verification for Hardware Security

Mengjia Yan

Spring 2026

Slides credit: Sharad Malik (Princeton)

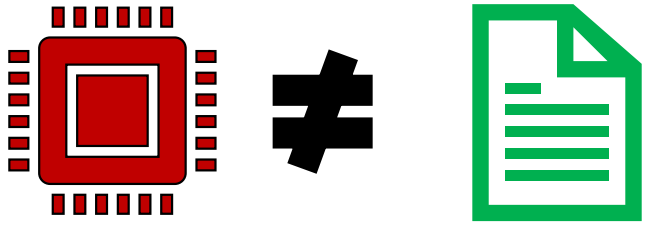
Cost of Advanced Designs



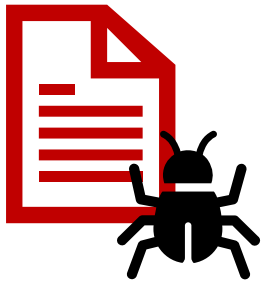
In many contexts, the term verification can be used in a loose way.

Source: IBS

Recall Hardware Bugs

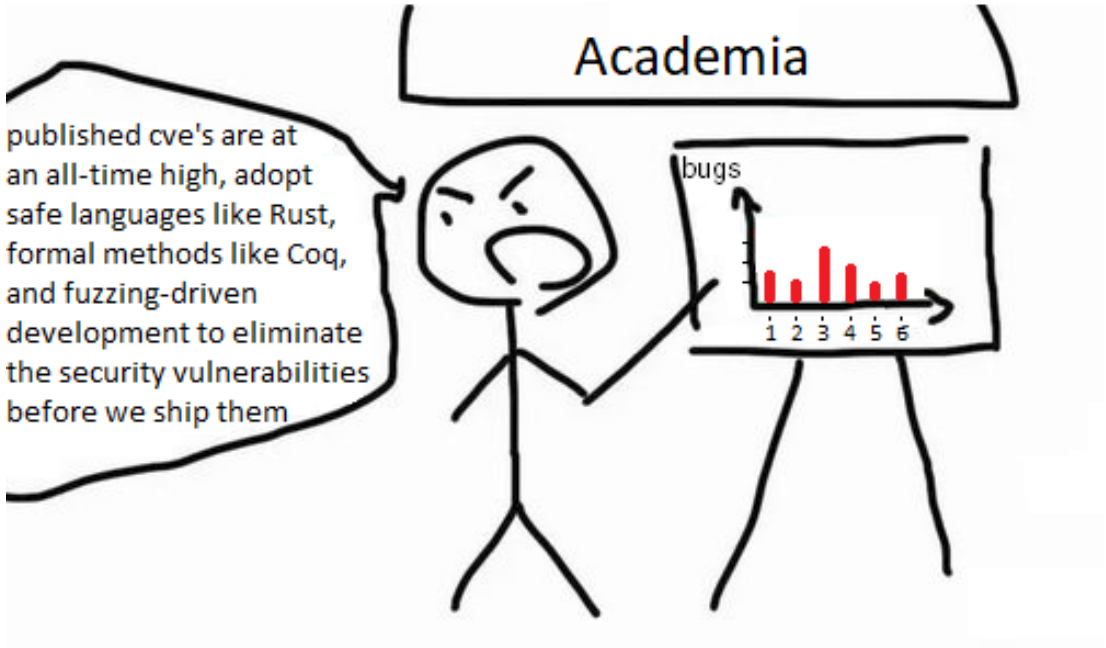


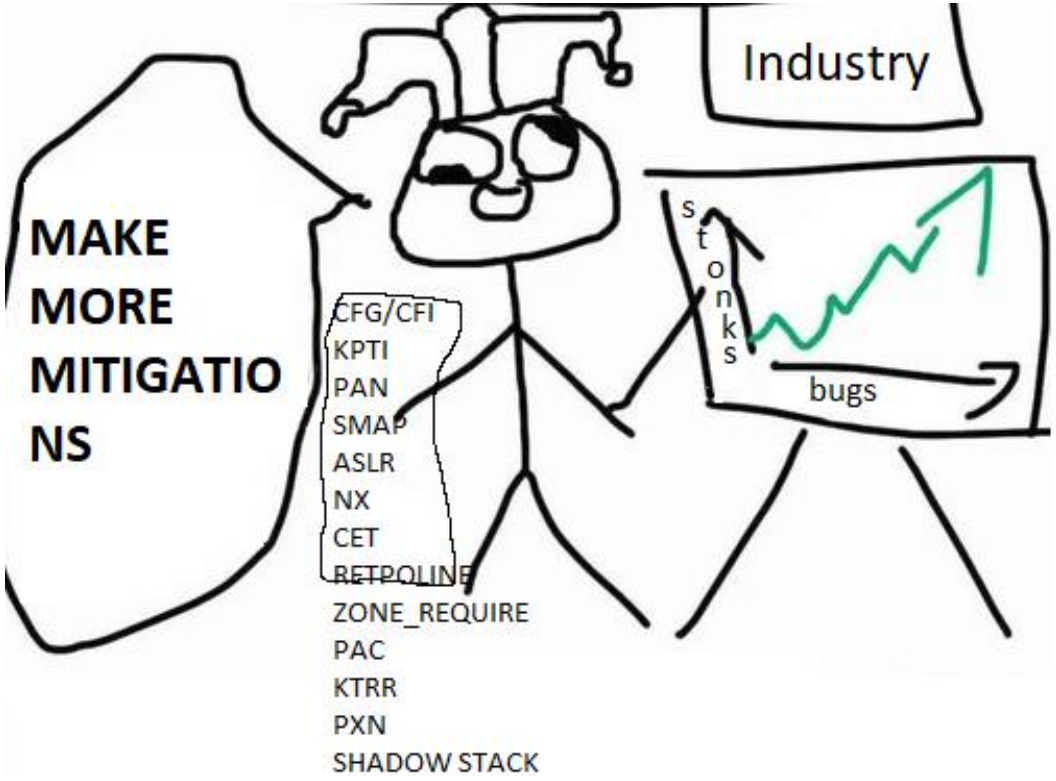
Implementation does not match specification (Errata)



Bugs in the specification

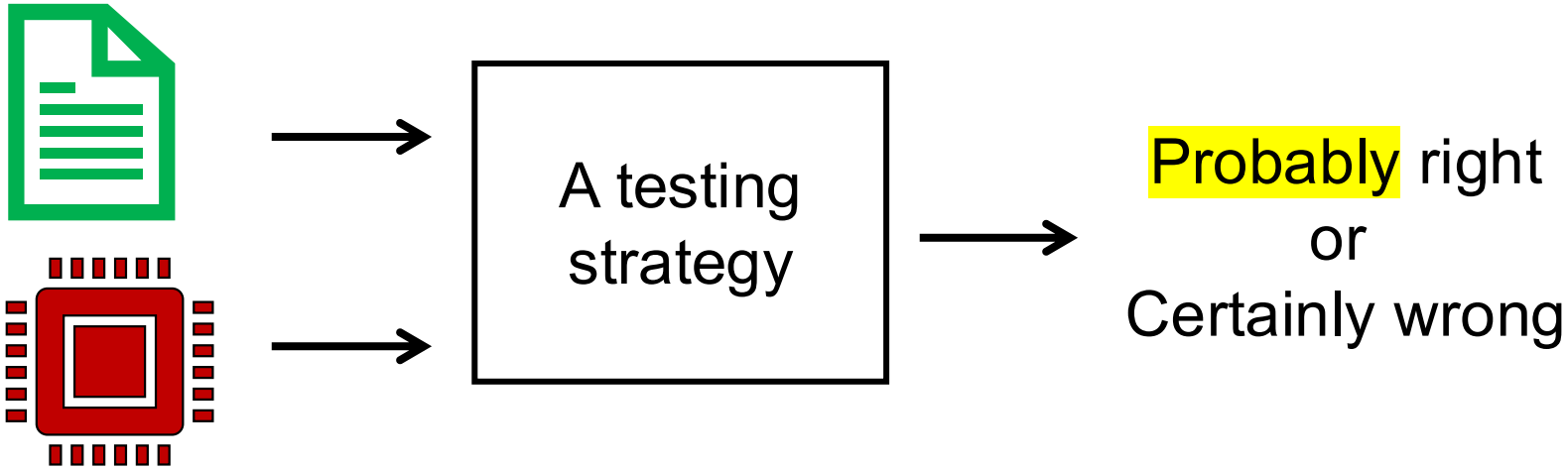
Under specification





https://twitter.com/gf_256/status/1321677851633029120/

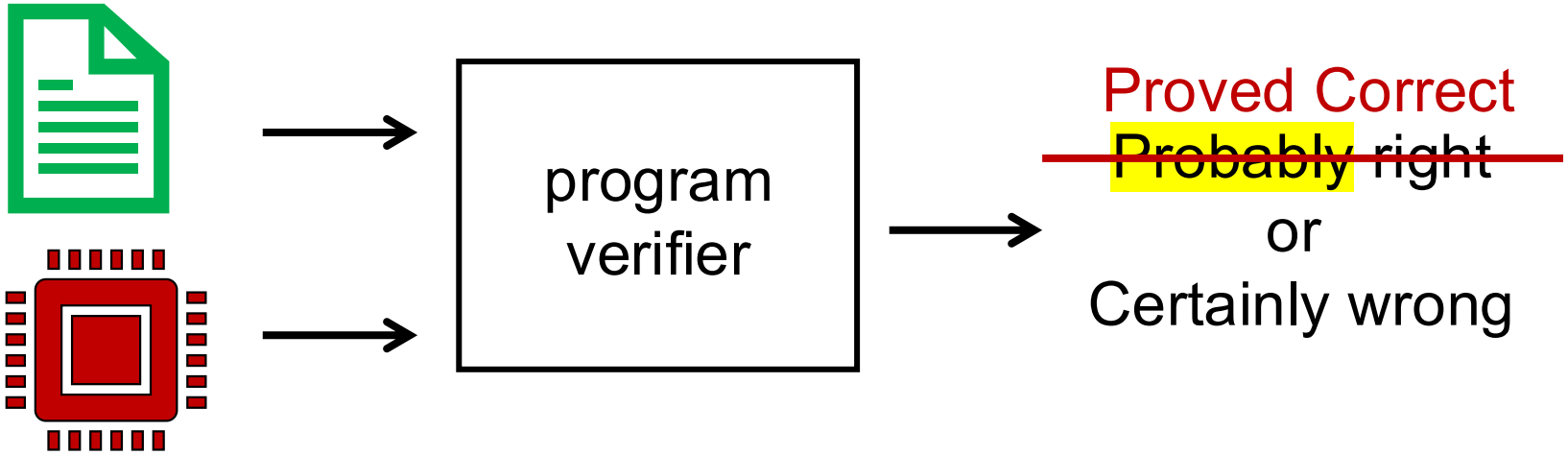
Program/Design Testing



- In practice: Test cases are generated to cover ***some* (not *all*)** inputs/statements/branches/paths etc.
- In principle: ***Exhaustive*** testing can prove correctness

> *Program testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence.*
>
> – Edsger Dijkstra

Program/Design Verification

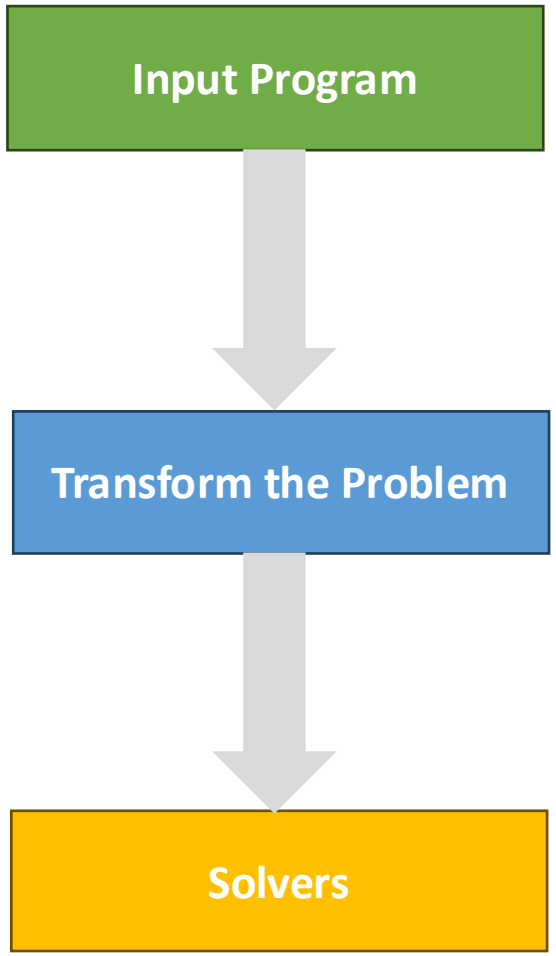


"Verification": formally **prove** that the program/design is correct

- Rigor: uses well established mathematical foundations
- Exhaustiveness: considers all possible program behaviors
- Automation (not all): uses computers to verify programs!

Overall, it is a search problem…

How does formal verification work?



Software (Linux kernel, crypto libraries); Hardware (Verilog RTL)

Symbolic execution, model checking, etc.

(! (= a (* 2 (+ 10 b)))))

SAT, SMT, BDDs, proof systems, etc.

Transform a Program via Symbolic Execution

C code:

```
int hash(int z){
   return (z+10)*2;
}

int test(int x, int y)
{
   if (x==hash(y))
      assert(false);
   return 1;
}
```

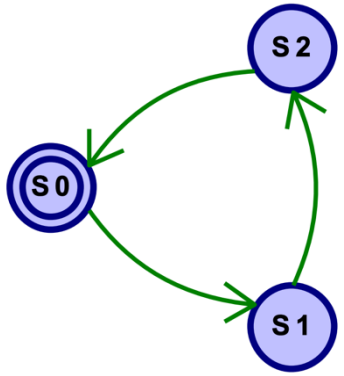
How will fuzzing behave to find this error?

Proof goal:

$$\forall, \boldsymbol{a}, \boldsymbol{b}, \qquad \boldsymbol{a} \neq (\boldsymbol{b} + \mathbf{10}) \times \mathbf{2}.$$

Transform Verilog Code to AIG

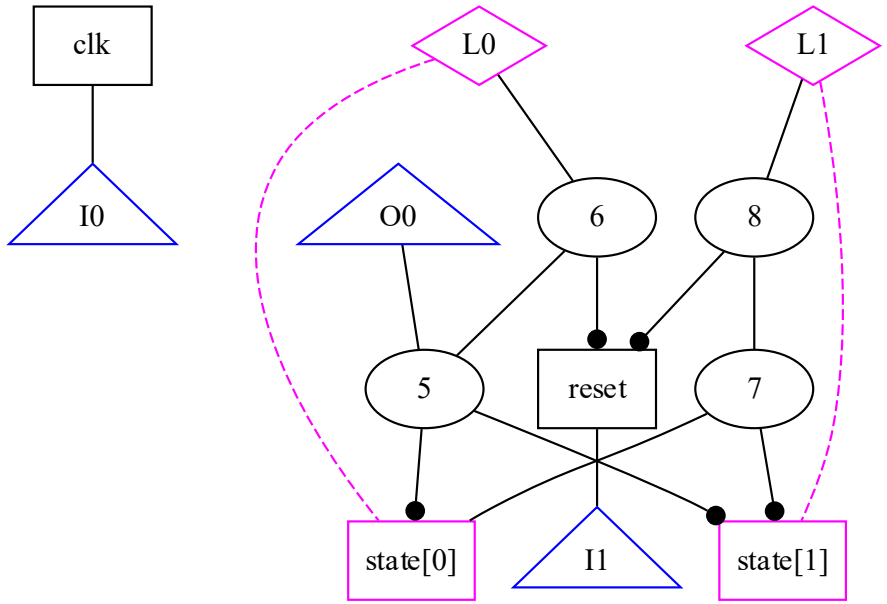
- AIG: And-Inverter Graph
  - A circuit representation built from only **2-input AND nodes** and **inverted edges** for NOT



A divide-3 FSM

```
module divideby3FSM (input clk, input reset, output q);
    reg [1:0] state, nextstate;

    always @ (posedge clk) // state register
        if (reset) state <= 2'b00;
        else state <= nextstate;

    always @ (*) // next state logic
        case (state)
        2'b00: nextstate = 2’b01;
        2’b01: nextstate = 2’b10;
        2’b10: nextstate = 2’b00;
        default: nextstate = 2’b00;
    endcase

    assign q = (state == 2'b00); // output logic
endmodule
```

```
module divideby3FSM (input clk, input reset, output q);
    reg [1:0] state, nextstate;

    always @ (posedge clk) // state register
        if (reset) state <= 2'b00;
        else state <= nextstate;

    always @ (*) // next state logic
        case (state)
        2'b00: nextstate = 2’b01;
        2’b01: nextstate = 2’b10;
        2’b10: nextstate = 2’b00;
        default: nextstate = 2’b00;
    endcase

    assign q = (state == 2'b00); // output logic
endmodule
```



How does formal verification work?

Input Program

Software (Linux kernel, crypto libraries); Hardware (Verilog RTL)

Transform the Problem

Symbolic execution, model checking, etc.

```
(! (= a (* 2 (+ 10 b)))))
```

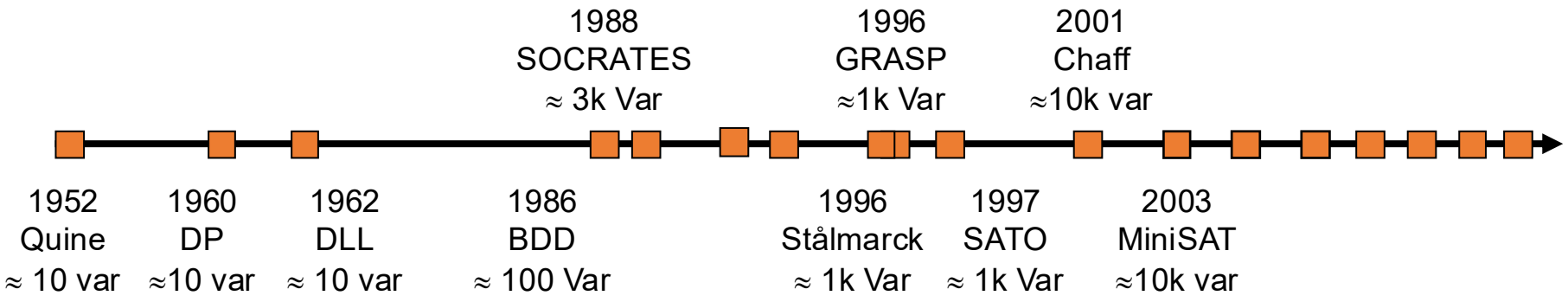
Solvers

SAT, SMT, BDDs, proof systems, etc.

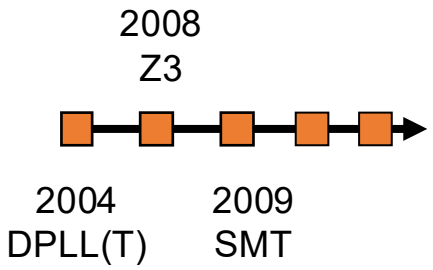
Success with SAT is at the heart of formal reasoning about systems.

Big Advancements in the Past Decade

(1) SAT: is a Boolean formula f satisfiable?



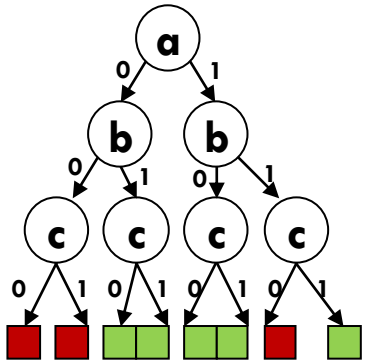
(2) SMT (Satisfiability Modulo Theory): is a first-order logic formula theory-satisfiable?



SAT in a Nutshell

F = (a + b)(a' + b' + c)

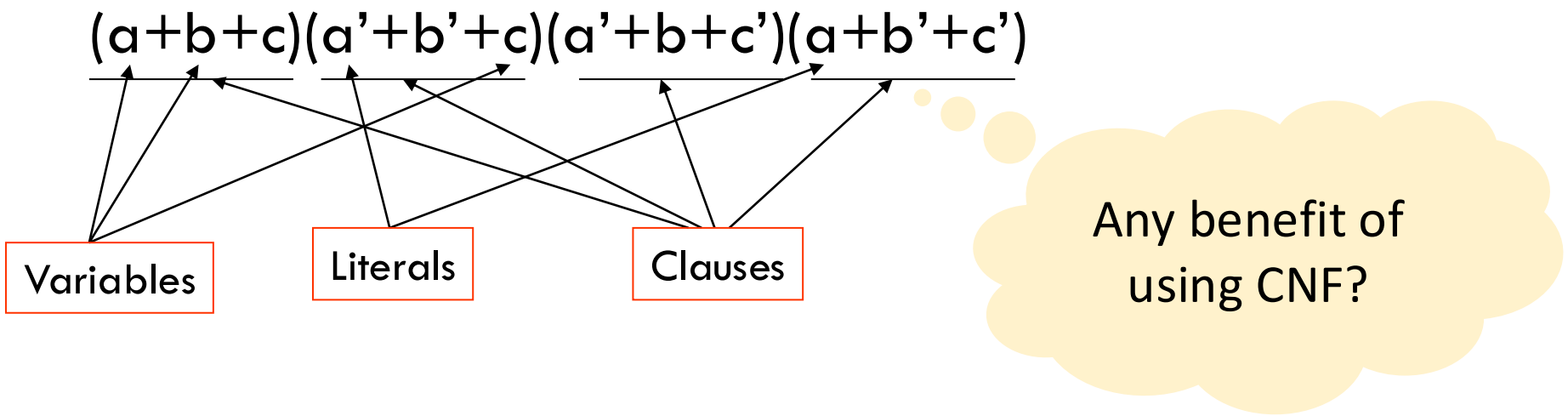
- Goal:
  - Given a propositional logic (Boolean) formula
  - Find a variable assignment such that the formula evaluates to 1
  - or prove that no such assignment exists.
- Naïve search algorithm: For *n* variables, there are 2^n possible truth assignments to be checked.
- First established NP-Complete problem
- But often ***tractable in practice*** *(millions of variables)*



S. A. Cook, The complexity of theorem proving procedures, *Proceedings, Third Annual ACM Symp. on the Theory of Computing*,1971, 151-158

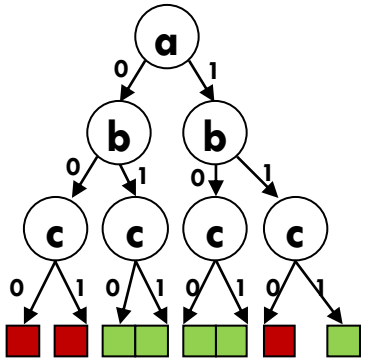
Problem Representation

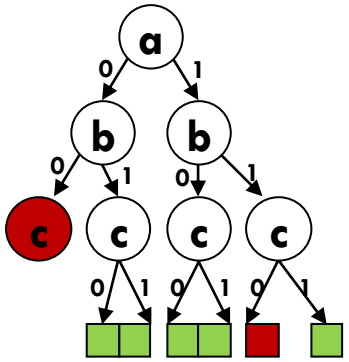
- Conjunctive Normal Form (CNF)
  - Representation of choice for modern SAT solvers
  - Decompose large problem into multiple smaller ones: every clause needs to be evaluated to TRUE
  - Algorithms take advantage of specific structure characteristics of the CNF constructed from real problems



Compare Naïve versus DLL [1962]

- Davis, Logemann and Loveland 1962 [DLL]
  - Backtrack exhaustive search for satisfying assignment

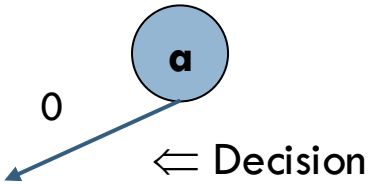




M. Davis, G. Logemann, and D. Loveland. A machine program for theorem-proving. *Communications of the ACM, 5:394–397, 1962*

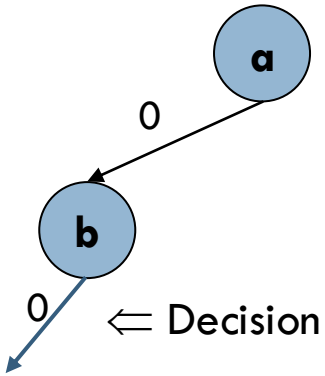
Basic DLL Search

→	(a' + b + c)
	(a + c + d)
	(a + c + d')
	(a + c' + d)
	(a + c' + d')
	(b' + c' + d)
→	(a' + b + c')
→	(a' + b' + c)



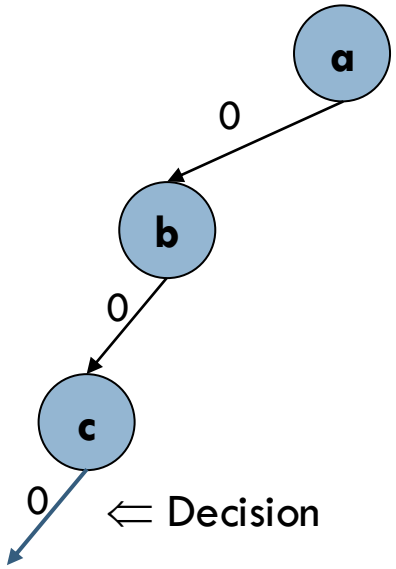
Basic DLL Search

(a' + b + c)
(a + c + d)
(a + c + d')
(a + c' + d)
(a + c' + d')
→ (b' + c' + d)
(a' + b + c')
(a' + b' + c)



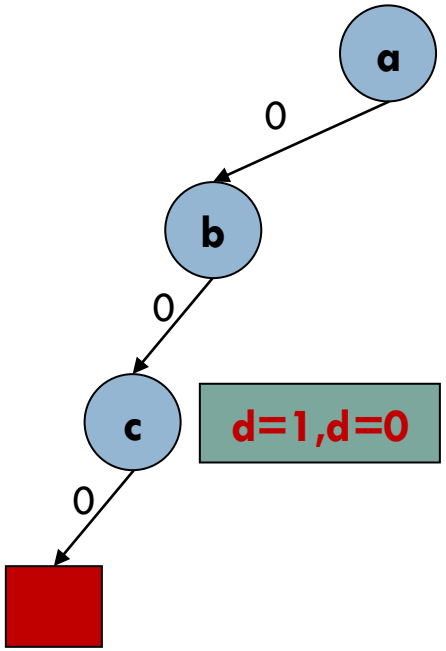
Basic DLL Search

	(a' + b + c)
	(a + c + d)
	(a + c + d')
→	(a + c' + d)
→	(a + c' + d')
	(b' + c' + d)
	(a' + b + c')
	(a' + b' + c)

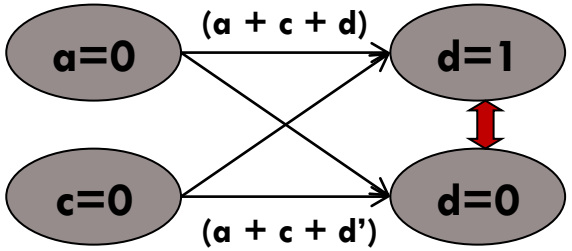


Basic DLL Search

(a' + b + c)
→ (a + c + d)
→ (a + c + d')
(a + c' + d)
(a + c' + d')
(b' + c' + d)
(a' + b + c')
(a' + b' + c)



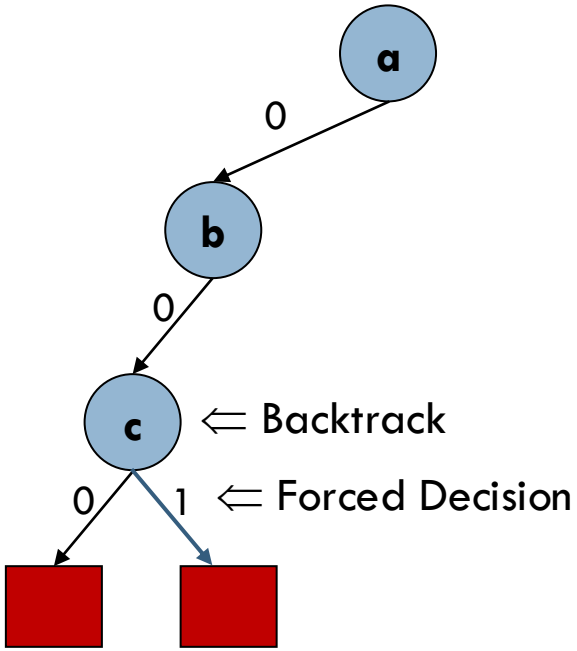
Implication Graph



Conflict!

Basic DLL Search

(a' + b + c)
(a + c + d)
(a + c + d')
→ (a + c' + d)
→ (a + c' + d')
(b' + c' + d)
(a' + b + c')
(a' + b' + c)



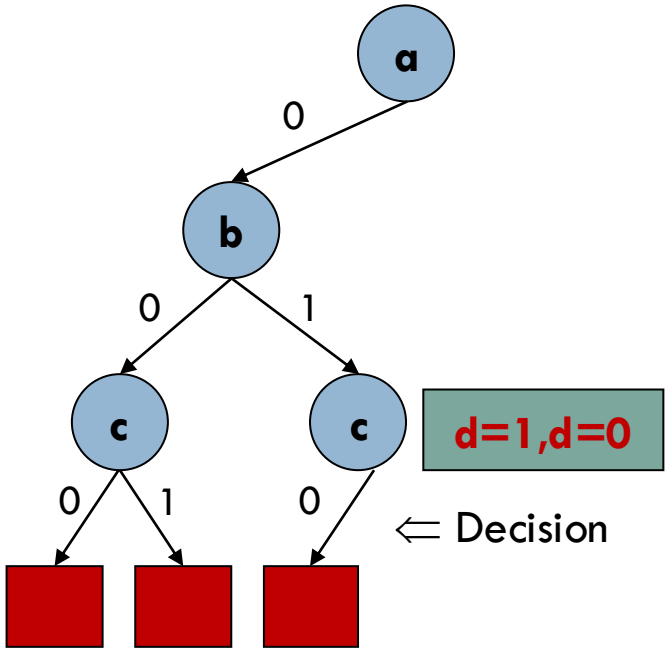
If we always search to the bottom, performance is as bad as naïve deep-first search.

What if I change the decision order to: a → c → d → b ?

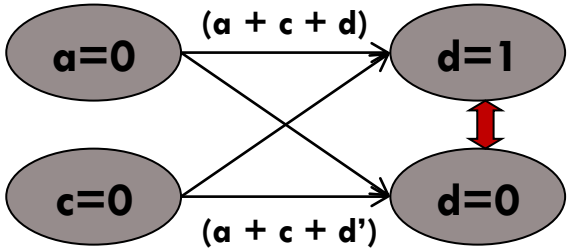
Practical problems have many such patterns to allow early backtracking.

Basic DLL Search

	(a' + b + c)
→	(a + c + d)
→	(a + c + d')
→	(a + c' + d)
→	(a + c' + d')
→	(b' + c' + d)
	(a' + b + c')
	(a' + b' + c)



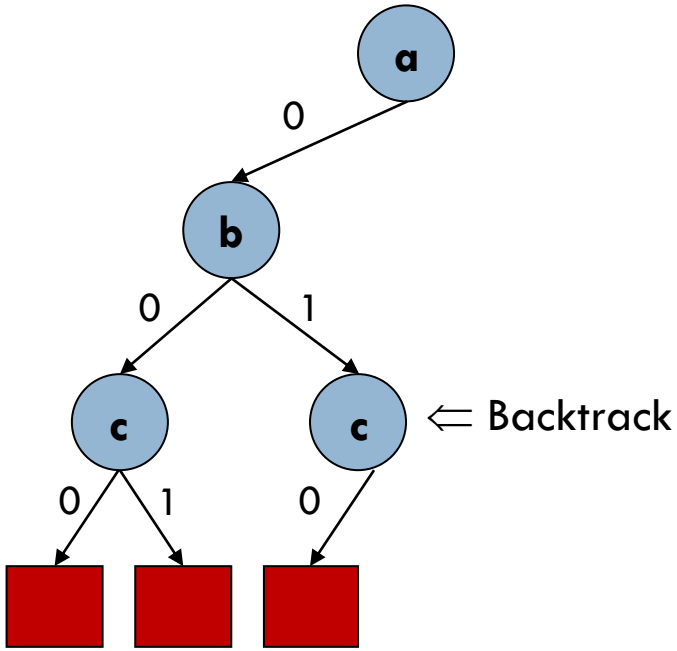
Implication Graph



Conflict!

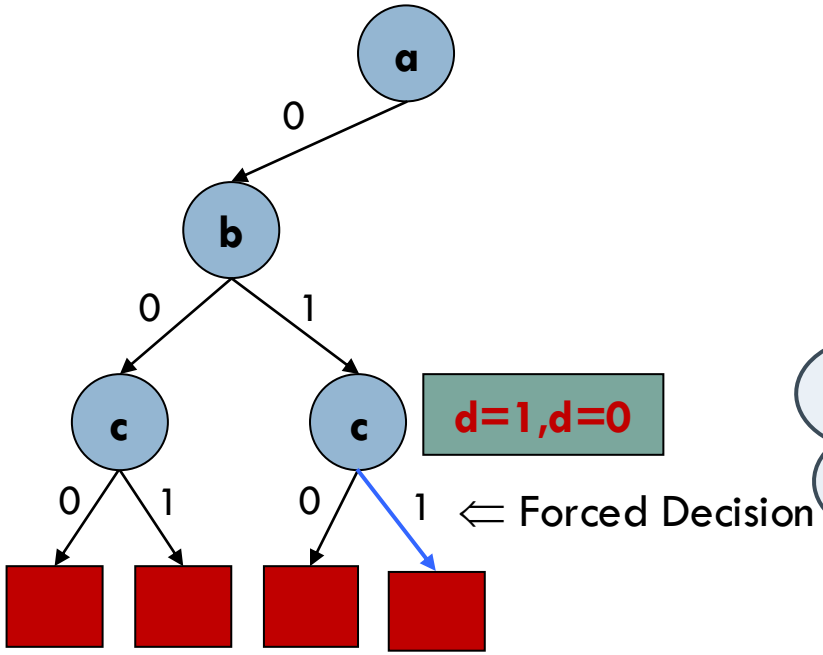
Basic DLL Search

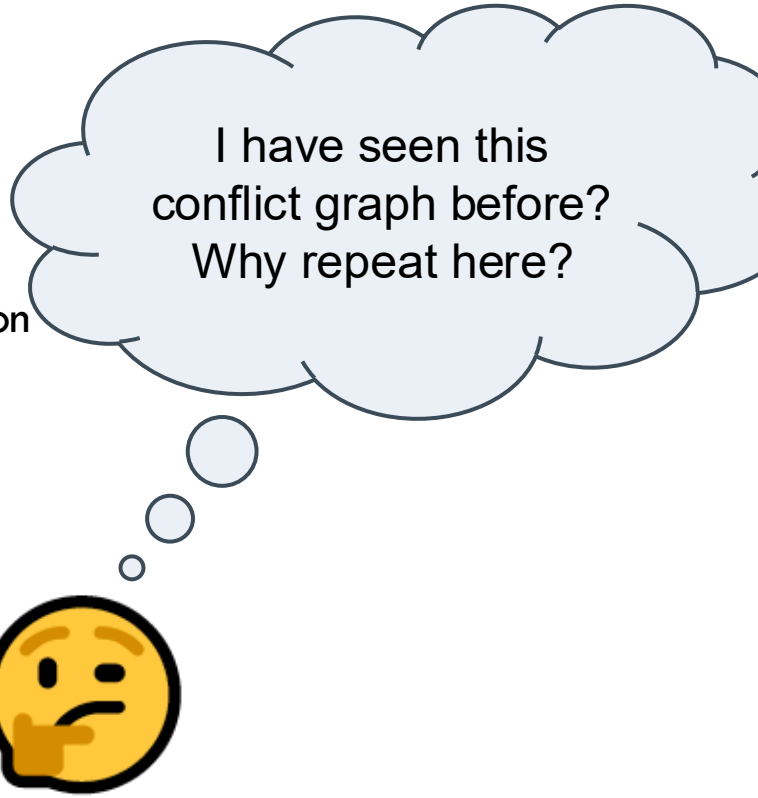
	(a' + b + c)
→	(a + c + d)
→	(a + c + d')
→	(a + c' + d)
→	(a + c' + d')
→	(b' + c' + d)
	(a' + b + c')
	(a' + b' + c)



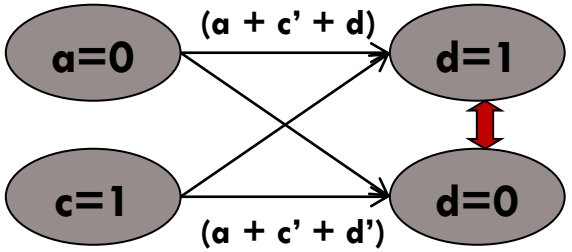
Basic DLL Search

(a' + b + c)
(a + c + d)
(a + c + d')
→ (a + c' + d)
→ (a + c' + d')
→ (b' + c' + d)
(a' + b + c')
(a' + b' + c)





Implication Graph



Conflict!

DLL versus CDCL

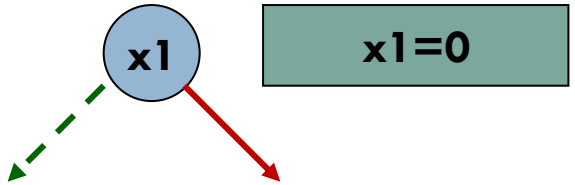
- DLL:
  - Search → Hit a dead end → backtrack one step

- CDCL: conflict driven learning and non-chronological backtracking
  - Search → hit a dead end → **figure out the reason of conflict**
  - → **remember** it so avoid hitting it again in the future
  - → backtrack to only **relevant** signals that can resolve the conflict

J. P. Marques-Silva and Karem A. Sakallah, "GRASP: A Search Algorithm for Propositional Satisfiability", *IEEE Trans. Computers*, C-48, 5:506-521, 1999.

Conflict Driven Learning and Non-chronological Backtracking

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'



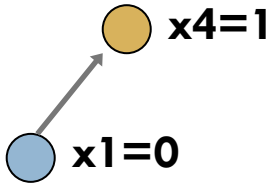
x1=0

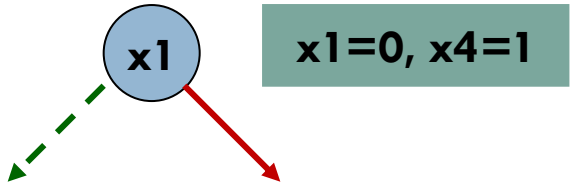
Red text means evaluated to 0, and green means evaluated to 1

For the graph on the left:
Blue circles means free variable, and brown circles mean inferred variable.
Edge describes the inferred relationship.

Conflict Driven Learning and Non-chronological Backtracking

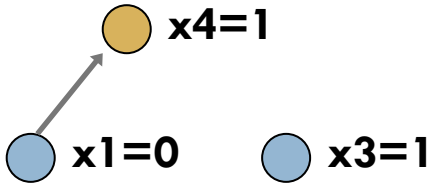
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

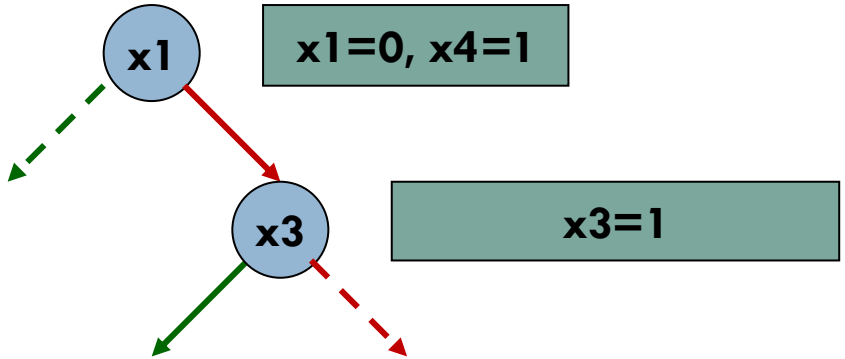




Conflict Driven Learning and Non-chronological Backtracking

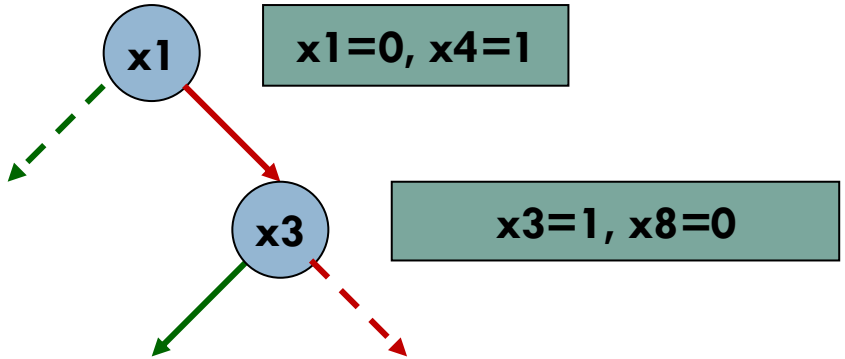
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

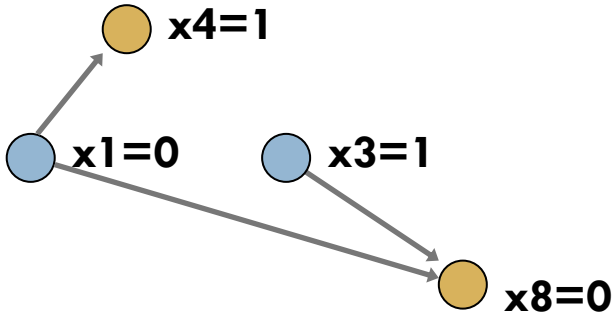




Conflict Driven Learning and Non-chronological Backtracking

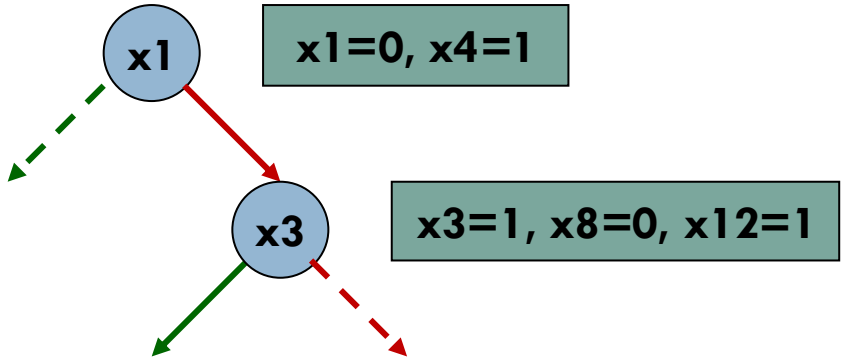
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

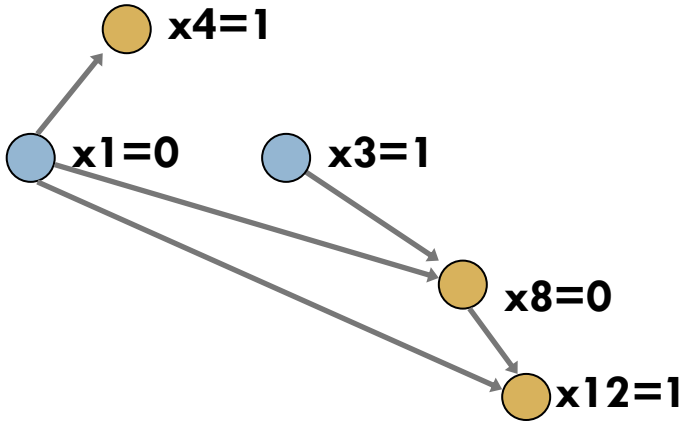




Conflict Driven Learning and Non-chronological Backtracking

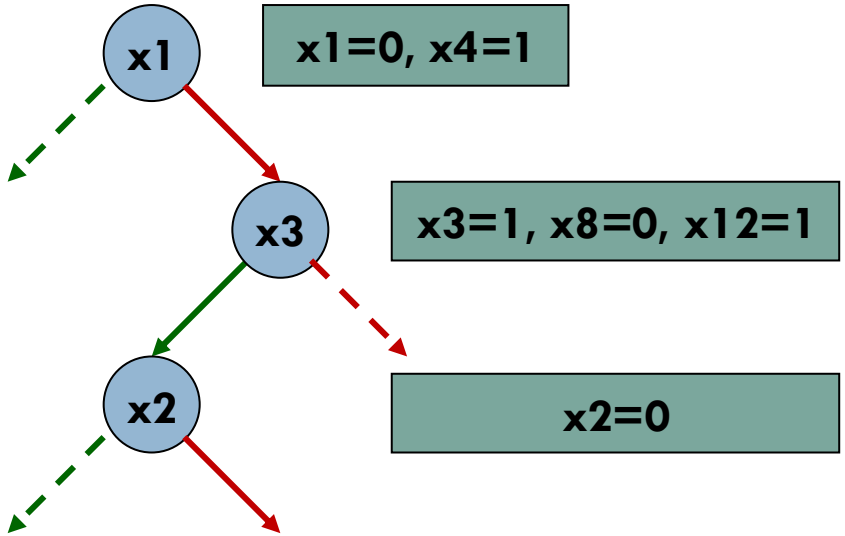
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

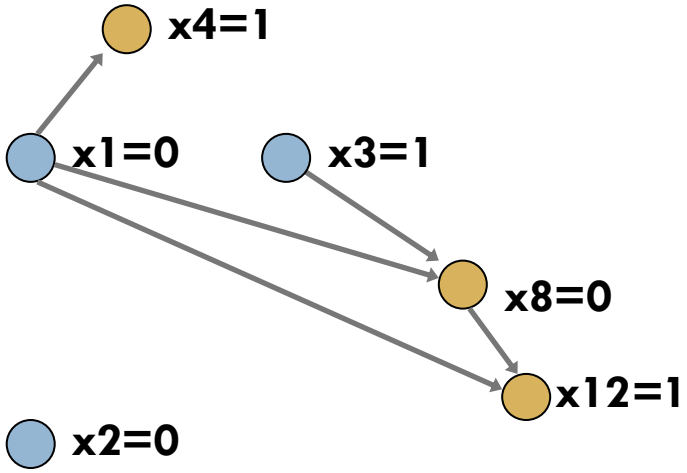




Conflict Driven Learning and Non-chronological Backtracking

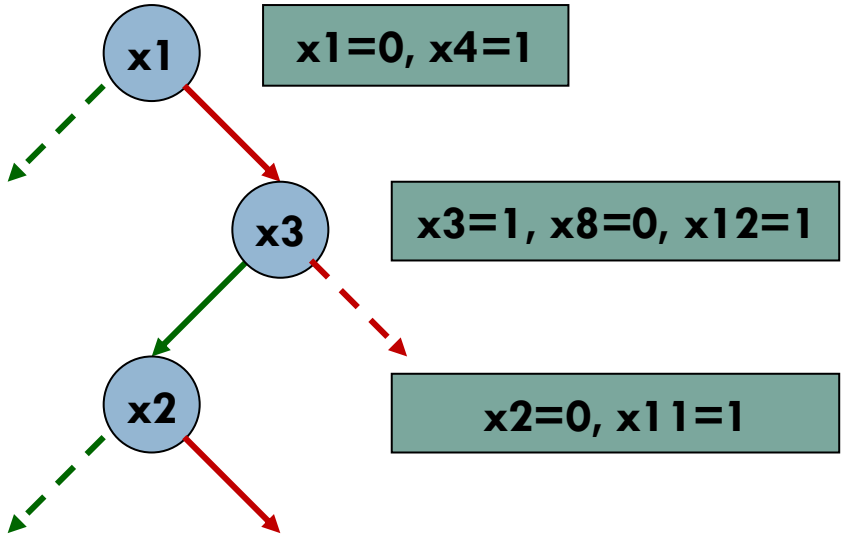
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

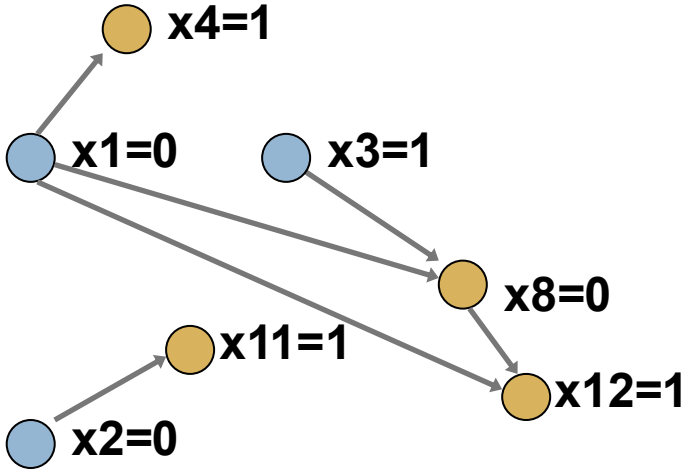




Conflict Driven Learning and Non-chronological Backtracking

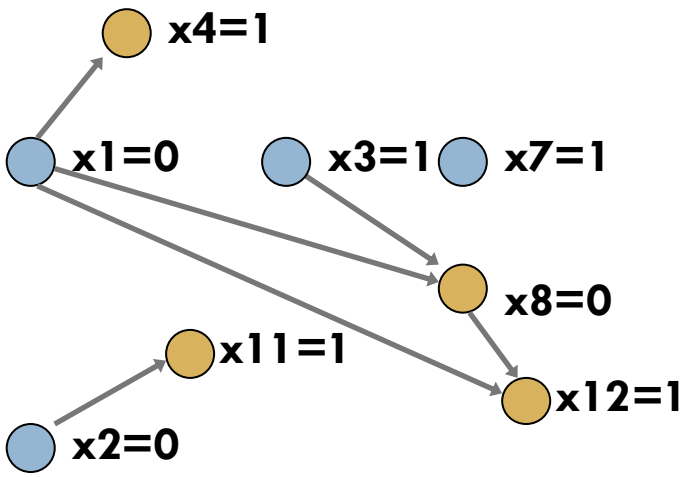
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

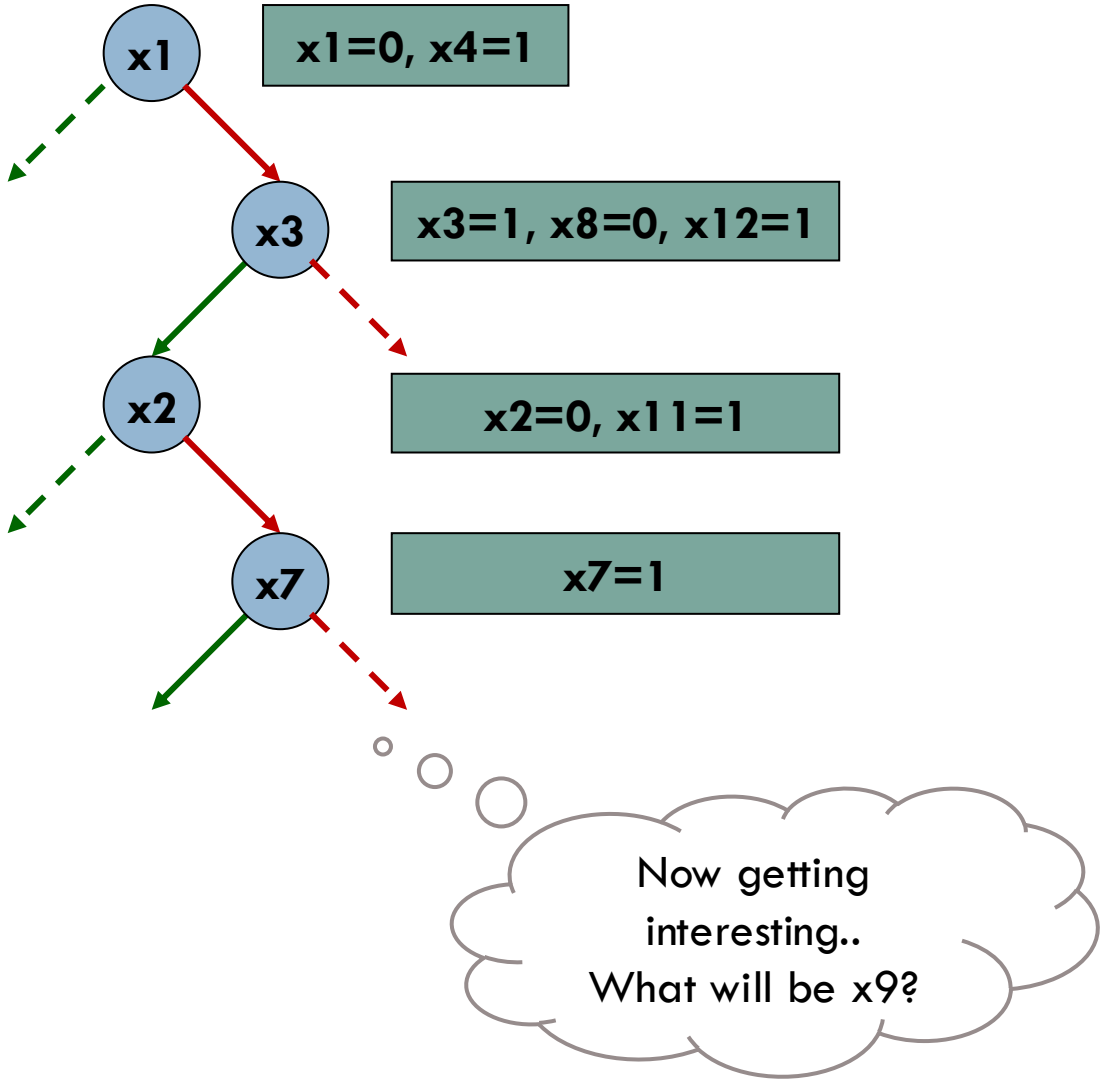




Conflict Driven Learning and Non-chronological Backtracking

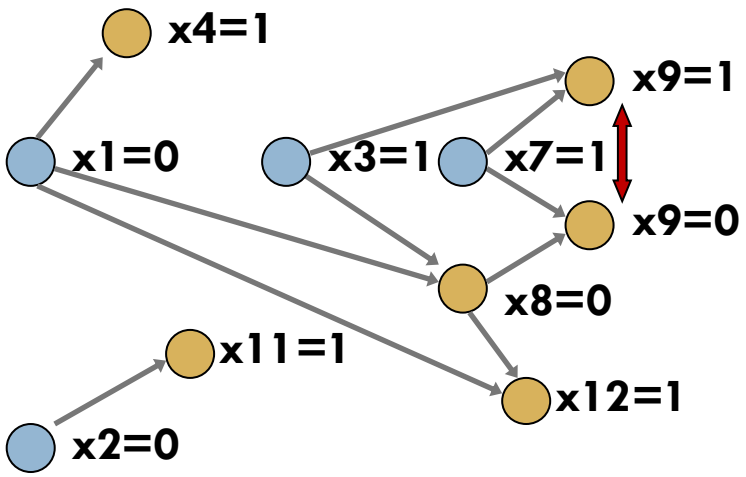
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

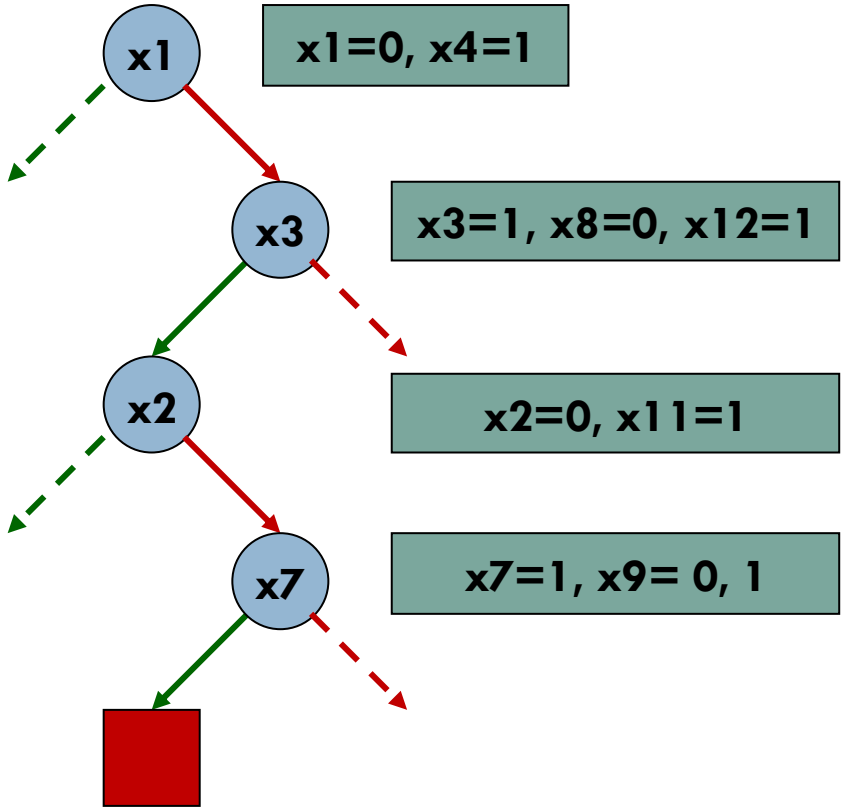




Conflict Driven Learning and Non-chronological Backtracking

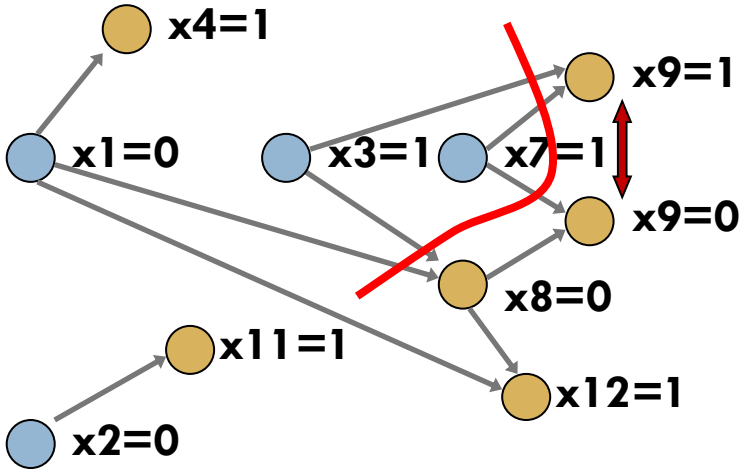
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

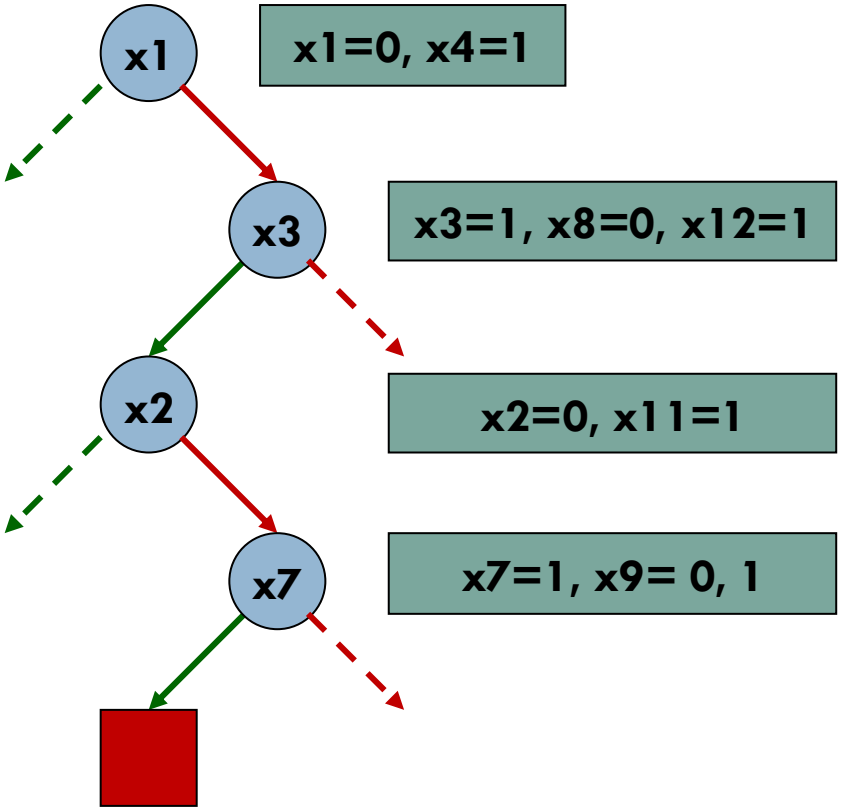




Conflict Driven Learning and Non-chronological Backtracking

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'



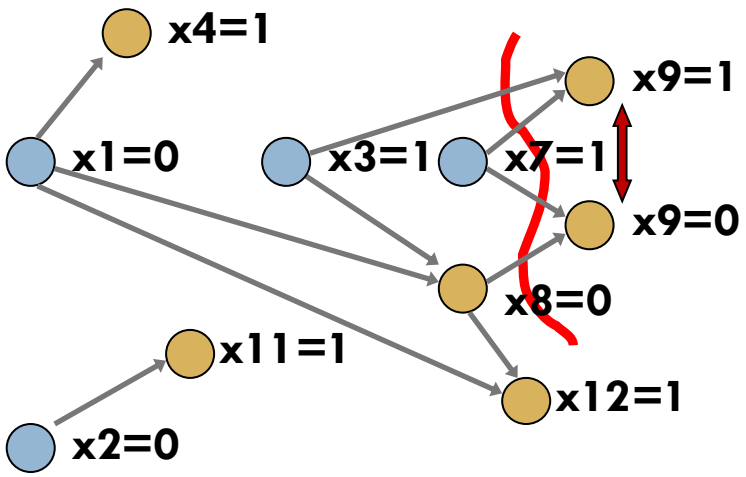


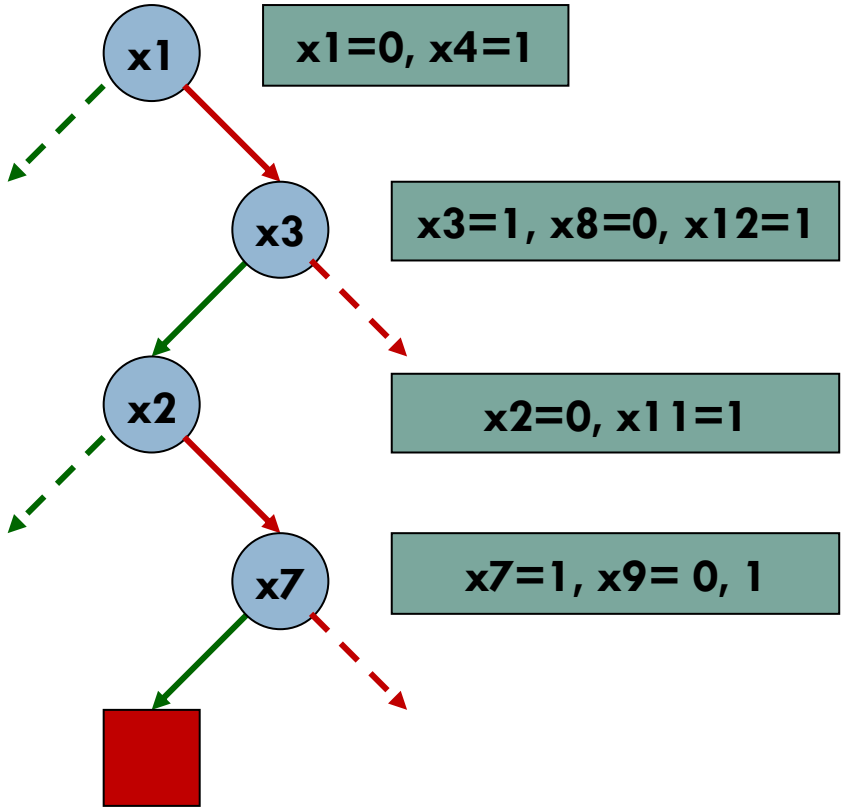
$x3=1 \wedge x7=1 \wedge x1=0 \rightarrow$ conflict

Add conflict clause: x3'+x7'+x1

Conflict Driven Learning and Non-chronological Backtracking

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'





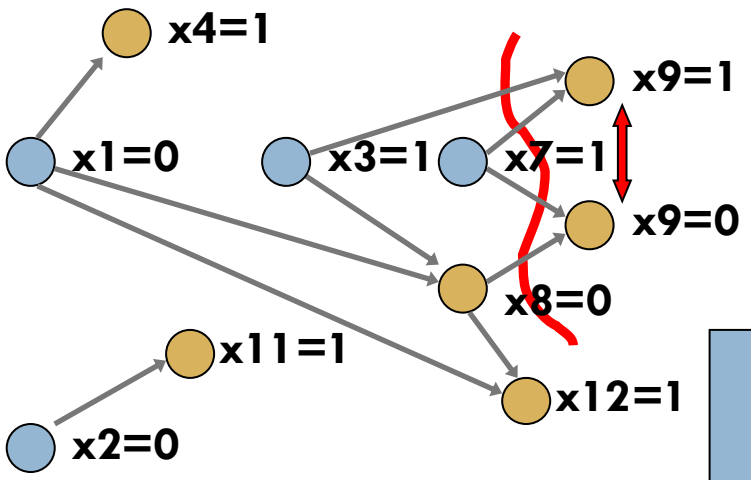
$x3=1 \wedge x7=1 \wedge x8=0 \rightarrow$ conflict

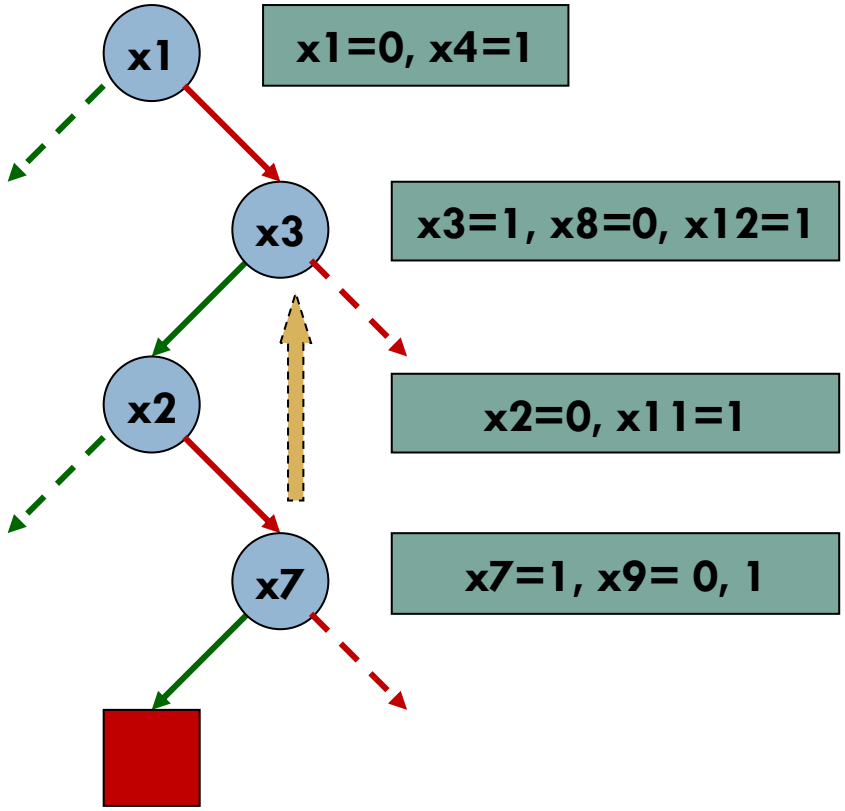
Add conflict clause: x3'+x7'+x8

Conflict Driven Learning and Non-chronological Backtracking

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

x3'+x7'+x8

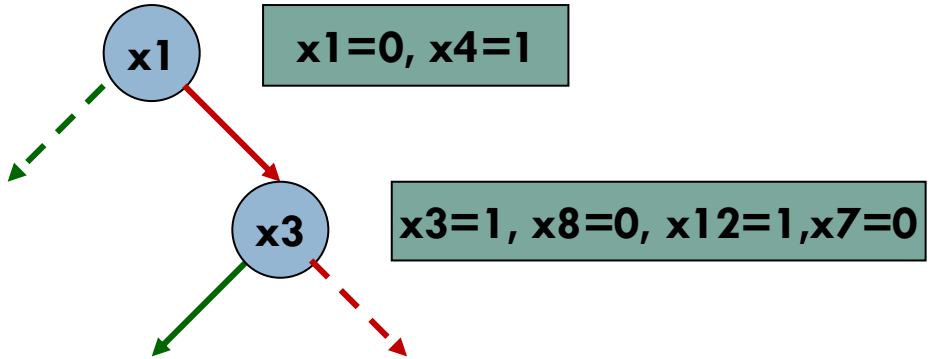


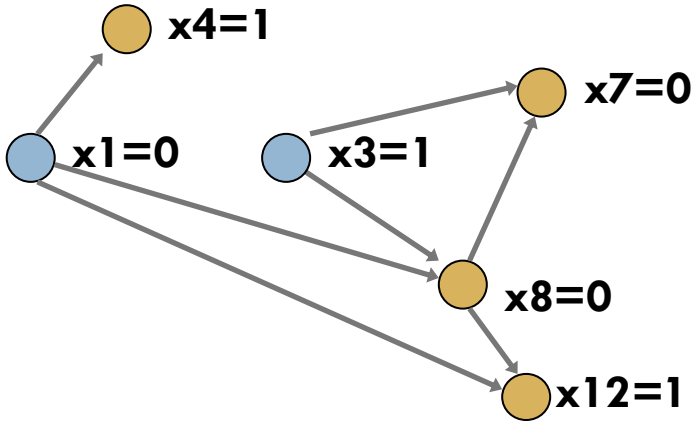


Backtrack to the decision level of x3=1

Conflict Driven Learning and Non-chronological Backtracking

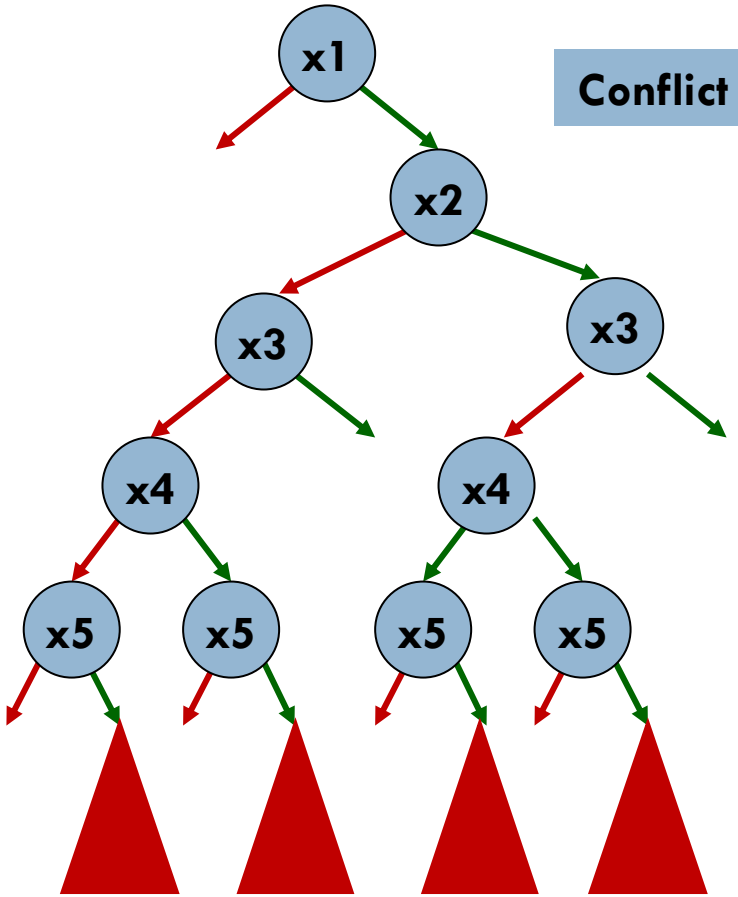
x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'
x3' + x7' + x8 ←new clause





Backtrack to the decision level of x3=1
Assign x7 = 0

What's the big deal?



Conflict clause: x1'+x3+x5'

Significantly prune the search space – learned clause is useful forever!

Useful in generating future conflict clauses.

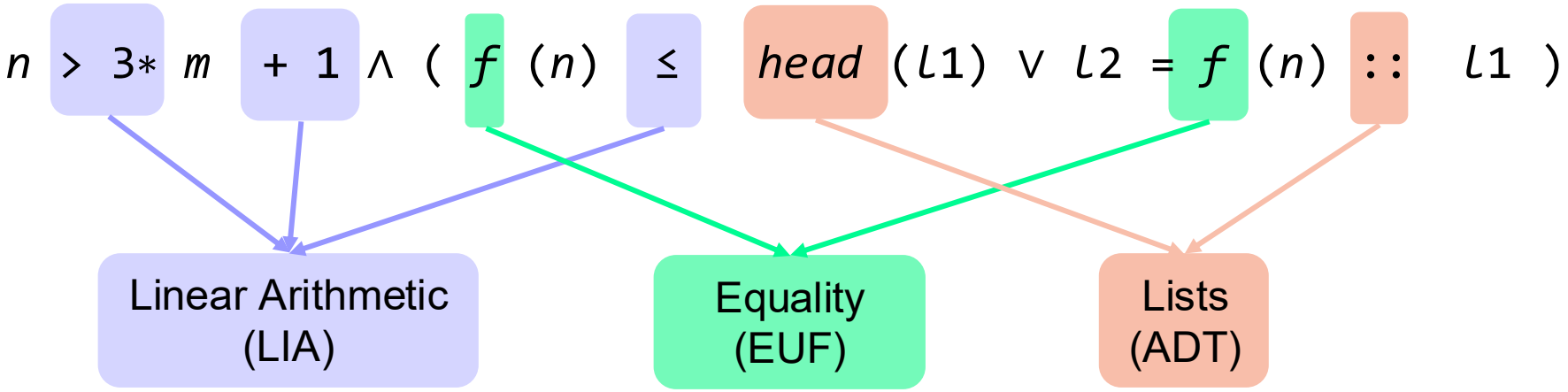
SAT Solvers: A Condensed History

- Deductive
  - Davis-Putnam 1960 [DP]
  - Iterative existential quantification by "resolution"
- Backtrack Search
  - Davis, Logemann and Loveland 1962 [DLL]
  - Exhaustive search for satisfying assignment
- Conflict Driven Clause Learning [CDCL]
  - GRASP: Integrate a constraint learning procedure, 1996
- Locality Based Search
  - Emphasis on exhausting local sub-spaces, e.g. Chaff, Berkmin, miniSAT and others, 2001 onwards
  - Added focus on efficient implementation
- "Pre-processing"
  - Peephole optimization, e.g. miniSAT, 2005

We covered these two algorithms to give you a taste of how the search works.

The Basic SMT Problem

- Determining the satisfiability of a logical formula with regards to some combination of background theories



Interacting with SAT/SMT Solvers

Interact with a solver

A counterexample is generated.
You can use it to fix your program.

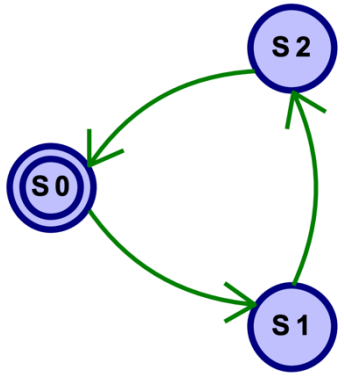
A proof is generated. Your program is bug-free!

(most of the time) …
Clueless. Basically the solver does not generate a result since the search cannot complete.

Need to consult other approaches, which require formal-method expertise:
Induction proof, find invariants, theorem proving, etc.
If interested, check out 6.512 https://frap.csail.mit.edu/main

Verifying Hardware Designs

- Hardware RTL code works as if a big loop



A divide-3 FSM

```
module divideby3FSM (input clk, input reset, output q);
    reg [1:0] state, nextstate;

    always @ (posedge clk) // state register
        if (reset) state <= 2'b00;
        else state <= nextstate;

    always @ (*) // next state logic
        case (state)
        2'b00: nextstate = 2’b01;
        2’b01: nextstate = 2’b10;
        2’b10: nextstate = 2’b00;
        default: nextstate = 2’b00;
    endcase

    assign q = (state == 2'b00); // output logic
endmodule
```

```
assert (state!=2’b11);
```

Toolchains to Verify Hardware

```
module divideby3FSM (input clk, input reset, output q);
    reg [1:0] state, nextstate;

    always @ (posedge clk) // state register
        if (reset) state <= 2'b00;
        else state <= nextstate;

    always @ (*) // next state logic
        case (state)
        2'b00: nextstate = 2’b01;
        2’b01: nextstate = 2’b10;
        2’b10: nextstate = 2’b00;
        default: nextstate = 2’b00;
    endcase

    assign q = (state == 2'b00); // output logic
endmodule
```

Verilog code

Compilation Toolchain
Yosys

CNF or AIG format

Model checking tools
(rIC3 or JasperGold)

An Example: Verify ISA Correctness

add

31-27	26-25	24-20	19-15	14-12	11-7	6-2	1-0
00000	00	rs2	rs1	000	rd	01100	11

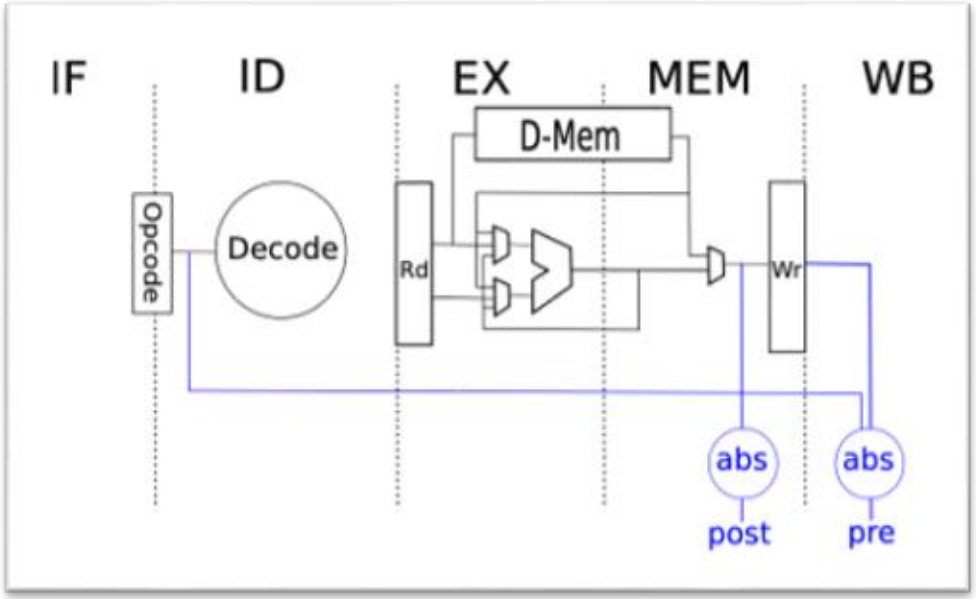
Format	add rd,rs1,rs2
Description	Adds the registers rs1 and rs2 and stores the result in rd. Arithmetic overflow is ignored and the result is simply the low XLEN bits of the result.
Implementation	x[rd] = x[rs1] + x[rs2]

RISC-V Instruction Set Specification

If interested, check x86 ISA specification for "add"

- Question 1: What assertion should we put into our RTL code?
- Question 2: If I have a 5-stage pipelined processor, when do I place the assertion?
- Question 3: If I want to catch some bypass bugs, how should I initialize the state of the processor?

A Tentative Plan



The instruction encoding below follows ARM ISA, different from RISCV from the last slide.

```
assign ADD_retiring = (pre.opcode & 16'b1111_1110_0000_0000) == 16'b0001_1000_0000_0000;
assign ADD_result = pre.R[pre.opcode[8:6]] + pre.R[pre.opcode[5:3]];
assign ADD_Rd = pre.opcode[2:0];

assert property (@(posedge clk) disable iff (reset_n)
ADD_retiring |-> (ADD_result == post.R[ADD_Rd]));
```

End-to-End Verification of ARM® Processors with ISA-Formal; Reid et al.; CAV'16

A Problem: Register Renaming

- A performance optimization to resolve WAW (write-after-write) data dependency

```
addi r1, r1, 4
ld   r2, 0(r1)
addi r1, r1, 4
ld   r3, 0(r1)
addi r1, r1, 4
ld   r4, 0(r1)
```

→

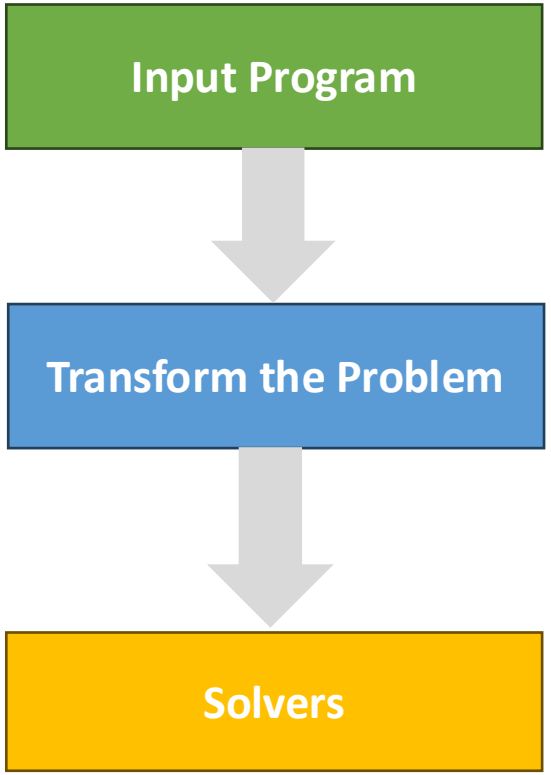
```
addi r1, r1, 4
ld   r2, 0(r1)
addi r11, r1, 4
ld   r3, 0(r11)
addi r12, r11, 4
ld   r4, 0(r12)
```

- Modern out-of-order processors do register renaming on-the-fly
  - Many different implementations, check out 6.823/6.5900
- Problem: How do we verify such processors?

Shadow logic to implement correct renaming logic

Summary

- Formal Verification: rigor, exhaustiveness, automation



For hardware verification: often needs domain expertise to translate specification to assertions

CNF/AIG

SAT solver is at the core for the performance

Next: Recitations

Hardware Formal Verification Toolchains